

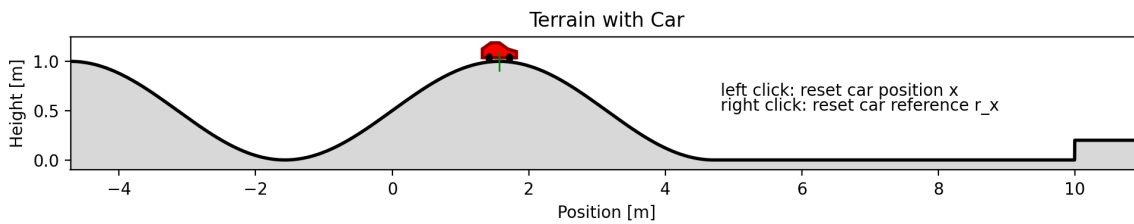
Prof. Dr. Stefan Leutenegger

Teaching Assistants: David Rohr, Alexander Hansson, Leonard Freissmuth, Hanzhi Chen

Exercise 10: PID & Model Predictive Control

Session date: 12/05/2026

Consider a car moving over hills as illustrated in the figure below. In this exercise, you will implement two controllers, a Proportional-Integral-Derivative (PID) controller and a Model-Predictive Controller (MPC) to control the car's position, e.g. to stabilize it on top of a hill, or to stop closely in front of the wall on the right without crashing.



The hills here are very specific and their height profile is

$$h(x) = \begin{cases} a \sin(x) & -\frac{3\pi}{2} \leq x \leq \frac{3\pi}{2} \\ 0 & \frac{3\pi}{2} < x \leq 10 \\ 0.25 & x > 10 \end{cases} \quad a > 0.$$

The car is of mass m and can be controlled with a forward facing force $u \in [F_{\min}, F_{\max}]$. Gravity further accelerates or decelerates the car as $-mg \sin(\alpha)$ tangentially to the curve, where $\alpha = \arctan\left(\frac{dh(x)}{dx}\right)$ denotes the local inclination. Apart from this and the gravity component in the up/down parts, the car also experiences friction that is modelled as $F_d = k_1 v + k_2 v|v|$, where $k_1 > 0$ and $k_2 > 0$. With this, we can approximate the car's dynamics:

$$\begin{aligned} \dot{x} &= v \cos \alpha(x) \\ \dot{v} &= -g \sin \alpha(x) - \frac{1}{m}(k_1 v + k_2 v|v|) + \frac{1}{m}u \end{aligned}$$

where x stands for the position along the horizontal axis and v for the velocity (tangential to the curve). For the flat part, you can simply set $\alpha(x) = 0$, for the hilly part we can expand the expressions to

$$\begin{aligned} \dot{x} &= \frac{v}{\sqrt{a^2 \cos^2 x + 1}}, \\ \dot{v} &= \frac{-ag \cos x}{\sqrt{a^2 \cos^2 x + 1}} - \frac{1}{m}(k_1 v + k_2 v|v|) + \frac{1}{m}u. \end{aligned}$$

Task 1: PID Position Control of Car Driving Over Hills

- i) Determine the equilibria of the so-called “autonomous” ($u = 0$), open-loop system, i.e. the states where $\dot{\mathbf{x}} = [\dot{x}, \dot{v}]^T = \mathbf{0}$? Perform a linear stability analysis to classify these equilibria as stable or unstable.

Hint:

$$\frac{d}{dx} \left(\frac{1}{\sqrt{\cos^2 x + 1}} \right) = \frac{\cos x \sin x}{(\cos^2 x + 1)^{1.5}}$$

- ii) Draw a block diagram of the PID position loop using x as the measured output. Implement the law directly in `E10_hill_driving.py` (e.g. in `PidController.control`), using the setpoint r_x , measured position x , and the controller step interval already defined in the simulator (e.g. $\Delta t = 1/\text{FPS}$). Start with $k_i = 0$, then tune k_p and k_d until the closed-loop behavior is satisfactory. Briefly discuss: can you regulate the car to a constant position on the slope? Can you come to rest near the far wall without overshooting? In what situations does integral action help, and what are its drawbacks here?
- iii) Sketch a modified architecture that adds a feed-forward path aimed at keeping v approximately constant along the slope (this is a relaxed notion of feed-forward: you may use a speed measurement $\tilde{v}$ and an estimate of the local inclination $\tilde{\alpha}$). Give the feed-forward force u_{ff} and implement it in `E10_hill_driving.py` alongside the PID loop. In a few bullet points, explain why you would add this additional component.

Task 2: MPC Position Control of Car Driving Over Hills

Instead of feeding back with a handcrafted law, we now want to use linear MPC to control the car's position (full state reference $\mathbf{r}$), employing linearization again, but this time around the current x and $\bar{v} = 0$, i.e. $\bar{\mathbf{x}} = [x, 0]^T$. In addition, we implicitly choose the linearization point $\bar{u}$ such that we have $\dot{v}(\bar{x}) = 0$. To move to discrete time, we employ Euler-forward for simplicity, so by application of the chain rule, the matrices $\mathbf{F}$ and $\mathbf{G}$ of the linearized discrete time system (assuming a constant time increment Δt) simply become $\mathbf{F} = \mathbf{I} + \Delta t \mathbf{F}_c$ and $\mathbf{G} = \Delta t \mathbf{G}_c$.

Now consider the finite-horizon quadratic cost

$$J(\delta \mathbf{x}_0, \dots, \delta \mathbf{x}_N, \delta u_0, \dots, \delta u_{N-1}) = (\bar{\mathbf{x}} + \delta \mathbf{x}_N - \mathbf{r})^T \mathbf{P} (\bar{\mathbf{x}} + \delta \mathbf{x}_N - \mathbf{r}) + \sum_{k=0}^{N-1} [(\bar{\mathbf{x}} + \delta \mathbf{x}_k - \mathbf{r})^T \mathbf{Q} (\bar{\mathbf{x}} + \delta \mathbf{x}_k - \mathbf{r}) + R(\bar{u} + \delta u_k)^2]$$

and the control optimization problem

$$\begin{aligned} J^* = & \min_{\delta u_0, \dots, \delta u_{N-1}} J(\delta \mathbf{x}_0, \dots, \delta \mathbf{x}_N, \delta u_0, \dots, \delta u_{N-1}), \\ \text{s.t.} \quad & \delta \mathbf{x}_{k+1} = \mathbf{F} \delta \mathbf{x}_k + \mathbf{G} \delta u_k, \\ & \bar{x} + \delta x_k \leq 10, \quad k = 0, \dots, N-1, \\ & \bar{u} + \delta u_k \geq u_{\min}, \quad k = 0, \dots, N-1, \\ & \bar{u} + \delta u_k \leq u_{\max}, \quad k = 0, \dots, N-1, \\ & \bar{\mathbf{x}} + \delta \mathbf{x}_0 = \mathbf{x}. \end{aligned}$$

Assume a time discretization of $\Delta t = 0.05$ seconds, and a horizon of $N = 100$ (i.e. 5 s).

- i) Implement the missing parts of the MPC controller `MpcController.control` in `E10_hill_driving.py`. Use `qp solvers` to solve the QP online as stated above.
- ii) Tune $\mathbf{P}$, $\mathbf{Q}$, and R (choose diagonal matrices) and observe the behavior for different reference positions.
- iii) Briefly discuss: can you regulate to a constant position on the slope? Can you come to rest near the far wall without overshooting? What are the advantages and disadvantages of MPC compared to PID?